

IP 파운데이션 월드모델 연구 노트 970

젠 판본이 저장소에 없었다

앞 노트의 수를 낸 러너가 git 어디에도 없었다 --- 다시 써서 다시 돌리니 앞 3,644 중 3,640 이 비트로 살아났고, 되살린 통제로 판을 돌리니 판이 내려간다

Sweetspot 월드모델 연구

2026년 8월 15일

초 록

앞 노트(969)는 산출물 여섯 전부에 자기 러너의 sha256 을 도장으로 박았다. 그 판본은 git 어디에도 없다 — 객체 저장소의 blob 79,429 개 전량을 훑어 일치가 0 이다. 969 의 사전등록 F3 는 「산출물 sha ≠ 커밋 blob sha 면 실패」인데 969 는 F3 을 OK 로 적었다. 969 는 같은 사이클에서 남의 산출물 여섯을 그 자로 대조했고 자기 러너만 안 봤다. 이 노트는 그것을 고친다 — 그러나 복원이 아니다. 없어진 바이트는 되찾을 수 없으므로 다시 쓰고 다시 돌렸다. 없어진 것은 셋이었다: (i) audit_cells() 의 「조항 60」 키 · (ii) § H 사료(969 의 헤드라인인 고고학을 낸 코드) · (iii) § E 전후 대조(P7 의 자리 — 앞선 비평은 § H 만 짚었다). 커밋본으로 여섯을 다시 갈고 앞 3,644 를 전량 대조하니 3,640 이 비트 동일이고 다른 넷은 전부 러너 자기 줄 변호였다.¹ 수는 살아 있었다 — 그러나 그것은 사후에 아는 것이고, 대조하기 전까지 그 노트의 어느 수도 인용할 수 없었다. 결으로 다섯을 정정한다. (i) 「Holm 통과 불변은 재 봐야 아는 것이었다」는 과장이다 — 두 설정 상수가 닿는 도메인은 10 중 5 이고, 통과 집합 셋 중 둘은 $\Delta\rho = 0.000000$ 으로 구조상 동일이며 나머지 하나도 문턱에서 70–277 SE 다. 그 검사에는 검정력이 없었다. (ii) 셋째 통과는 문턱에서 1.88 SE 이고 969 자신의 기계가 씨앗을 바꾸면 뒤집힐 수 있다: true 로 표시했다. (iii) 개별 $\Delta\rho$ 를 두 이름 중 큰 쪽으로 골라 적었다 — 주 이름으로는 애니가 0.0072 가 아니라 -0.000598 이다. (iv) 「0.5 채움은 층이 셋」은 세 파일만 훑은 산물이다 — lab/ 전량이면 34 자리이고, 돌려서 세면 챔피언 판에서 도는 것은 14 줄이다. (v) harness.py:241 이 「WIKI_DROP 이 떨어지는 자리」라는 인용은 헤드라인 도메인에서 틀리다. 마지막으로 두 사이클이 미룬 실험을 했다: 설정 상수를 비운 채 챔피언 판 24 주행. $\Delta\rho = -0.002962 \pm 0.000973$ (짜지는 12 씨앗 · 3 / 12).

1. 무엇이 없어졌나

주장 1. 969 의 수를 낸 러너는 git 어디에도 없었다 — 사전등록 F3 가 발화했어야 했고 969 는 그것을 통과로 적었다

반증 조건.

969 의 산출물 여섯은 모두 code_stamp 를 싣고, 그 안의 runners/dropaudit969.py 항목이 cc4244b1... 다. 가지의 모든 커밋에 있는 그 파일은 9198f932... 이고, 객체 저장소 blob 79,429 개 어디에도 cc4244b1 은 없다. code_stamp 74 파일 중 73 이 커밋과 일치하고 어긋난 하나가 969 자기 러너다.

이 배치는 우연이 아니다. 969 는 같은 사이클에서 앞 노트의 자를 수리하며 「자는 젠 소스의 sha 를 산출물에 박는다」는 조항을 만들었고, 그 자로 남의 산출물 여섯을 대조해 통과시켰다. 자기 러너는 그 분모에 없었다. 도장을 찍는 자가 자기를 분모에서 빼면 그 조건은 항진명제가 된다.

주장 2. 없어진 것은 셋이다 — 조항 60 키 · § H 사료 · § E 전후 대조. 다시 써서 다시 돌리니 산출물 다섯의 앞

3,644 중 3,640 이 비트 동일이고, 여섯째에는 줄번호도 sha 도 아닌 진짜 차이(연 수 2,241 → 2,154)가 있다

반증 조건.

커밋본을 그대로 돌려 969 산출물과 옆별로 맞춰 보면 두 구멍이 나온다.

산출물	앞	다른 앞
out969_wiring	227	0
out969_drop	2,440	0
out969_prop	789	0 [†]
out969_game	67	0
out969_metacmp	121	4 [‡]
합계	3,644	4

[†] § E 를 다시 쓴 뒤. 커밋본은 prop 단계에서 § C · § D 만 내고 § E 를 아예 안 냈다. [‡] 넷 다 dropaudit969.py 의 자기 줄 번호다(893→:910 등) — 없어진 코드를 써 넣었으니 줄이 밀리는 것은 불가피하고 수가 아니다.

§ H 는 이 노트의 헤드라인인 고고학을 낸 코드다 — archive/pre-github 에서 설정 상수의 도입 커밋을 찾고, 원장의 「...잘라내기」 항목과 노트 350 의 사전등록을 읽어 「이 상수는 왜 있나」에 답한다. § E 는 P7 의

자리다. 둘 다 없는 채로 커밋되어 있었고, 그 사실은 산출물을 다시 돌리기 전에는 보이지 않는다.

2. 결으로 정정하는 다섯

주장 3. 「Holm 통과 불변」은 재 봐야 아는 것이 아니었다 — 그 강건성 검사에는 검정력이 거의 없었다

반증 조건.

969 는 설정 상수를 비우고 다시 재도 Holm 통과 수가 두 이름 모두 불변임을 보이고 「이것은 재 봐야 아는 것이었다」고 적었다. §E 를 다시 읽으면 그렇지 않다. 두 상수가 닿는 도메인은 10 중 5(도서 · 시장팝업 · 애니 · 웹툰 · 팝업)이고 나머지 다섯은 $\Delta\rho = 0.000000$ 으로 구조상 동일이다. 그런데 통과 집합은 세계애니 · 애니 · 게임이고 그중 세계애니와 게임은 안 닿는 쪽이다. 움직일 수 있는 유일한 통과인 애니도 문턱에서 70.53 SE(들뜸) · 276.8 SE(긴 띠) 떨어져 있다. 통과 수는 달리 나올 수가 없었다.

전후를 신는 것은 여전히 옳다. 다만 「불변이었다」를 증거로 쓰려면 그 검사가 무엇을 뒤집을 수 있었는지를 같이 적어야 한다.

주장 4. 셋째 통과는 문턱에서 1.88 SE 이고, 969 자신의 기계가 「씨앗을 바꾸면 뒤집힌다」로 표시했다

반증 조건.

969 의 초록은 「문턱 근처가 아니어서 결론이 안 바뀐 것」이라 적었다. 사전등록이 정한 50,000 뽑기에서 게임/들뜸은 $p = 0.0056199 \cdot \text{Holm}$ 문턱 0.00625 · MC SE 0.0003343 이라 1.88 SE 이고, 같은 산출물의 필드가 씨앗을 바꾸면 뒤집힐 수 있나: true 다. 그 통과가 「3」을 만드는 통과다. 969 는 별도로 200,000 뽑기 게임 팔을 돌려 그 결정을 굳혔지만(4.44 SE 통과), 그것은 다른 분모의 다른 주행이고 명제 절의 「통과 3」은 50,000 뽑기의 수다.

주장 5. $\Delta\rho$ 를 두 이름 중 큰 쪽으로 골라 적고 어느 이름 인지 밝히지 않았다

반증 조건.

969 는 개별 ρ 의 이동을 「팝업 0.0676 · 도서 0.0496 · 시장팝업 0.0138 · 애니 0.0072 · 웹툰 0.0003」로 적었다. §E 는 두 이름을 갈라 낸다:

도메인	긴 띠 $\Delta\rho$ (주 이름)	들뜸 $\Delta\rho$
애니	-0.000598	+0.007213

969 가 인용한 수는 둘 중 절댓값이 큰 쪽이다. 사전등록이 정한 주 이름은 긴 띠이고, 그 이름으로는 애니가 -0.000598 이라 969 의 예측(「애니가 0.01 이상 움직인다」)이 한 자릿수 더 크게 틀렸다.

주장 6. 「0.5 채움은 층이 셋」은 세 파일만 훑은 산물이다 — 돌려서 세면 챔피언 판에서 도는 것은 14 줄이고,

harness.py:241 인용은 헤드라인 도메인에서 틀리다
반증 조건.

969 의 그 표는 자기 러너가 `wikiaxes.py·harness.py·forms.py` 세 파일만 하드코딩으로 `grep` 한 결과다 — `lab/loop.py` 는 0.5 로 한 번도 안 훑었다. `lab/` 전량으로 넓히면 채움 호출은 34 자리 · 19 파일, 0.5 리터럴 전량은 94 자리 · 34 파일이다.

정적 세기는 「있다」만 말한다. `np.full/np.where` 를 감싸 채움값이 0.5 인 호출의 줄을 세는 파수병을 달고 자료 짓기와 챔피언 한 주행을 덮으면 14 / 34 만 실제로 돈다. 상위 몇 줄: `forms.py:163(864 회) · harness.py:241(289) · trendaxes.py:54(80) · calaxes.py:127(66) · wikiaxes.py:192(40) · loop.py:612(31) · loop.py:683(19)`.

그 마지막 둘이 핵심이다. 헤드라인 도메인(팝업)은 `harness.py:241` 을 안 지난다 — 그 도메인이 `harness.PRIMARY` 라서 뒤에 오는 함수가 블록을 통제로 다시 짓고, 살아남는 0.5 는 `loop.py:612` 에서 온다. 969 가 감사한 18 칸 중 6 칸과 검색 상수 효과의 100% 가 그 줄을 비껴간다. 문서대로 :241 에만 계측기를 달면 주 도메인을 조용히 놓친다.

결: `forms.py:166` 은 챔피언 판에서 0 회 돈다 — 축 목록이 전 도메인 이름의 교집합이라 그 `else` 가지가 원리상 도달 불가다.

3. 두 사이클이 미룬 실험

주장 7. 설정 상수를 비우면 축 이름이 안 바뀐다 — 그래서 「되살린 통제로 판을 돌려라」는 물을 수 있는 물음이었고, 답은 「되살리면 판이 내려간다」다

반증 조건.

앞 노트는 이 실험의 전제 조건으로 「`AXIS_MODE="common"`」의 축 붕괴를 먼저 풀어야 한다」를 걸어 두었다. 그 전제는 틀렸다. 축 붕괴는 열을 뺄 때 생긴다 — 도메인마다 다른 열을 빼면 교집합이 무너진다. 설정 상수를 비우는 것은 열을 하나도 안 뺀다: 그 상수는 축 dict 에서 도메인을 빼고, 그러면 판 조립기가 이름을 남긴 채 값만 0.5 · 마스크 0 으로 채운다. 실측으로 현행과 비율에서 도메인별 축 이름 수가 완전히 같다(11 도메인 36 · 게임 40 · 교집합 36). 그래서 그대로 돌렸다 — 짝지는 12 씨앗 × 2 팔 = 24 주행:

기준선(현행)	0.470343
처리(두 상수 비움)	0.467380
$\Delta\rho$ (짝지는 C-B)	-0.002962
짝 SD	0.003370
짝 SE	0.000973
$ \Delta\rho \div$ 짝 SE	3.04
이긴 씨앗	3 / 12

기준선은 정본 0.47034 를 재현한다. 채택 크기는 이 저장소가 「못 정했다」로 남긴 양이므로 판정은 $\Delta\rho \pm \text{SE}$ 로만 적는다.

되살리면 판이 내려간다. 이것은 앞 노트가 켄 사료와 부호가 맞는다 — 노트 350 은 빼는 방향으로 판이 올랐다고 적었고 그래서 뺐다. 그러므로 「그 상수를 진짜로 지울 것인가」의 답은 「지우면 예측이 나빠진다」다. 주의 — 분모가 다르다 — 노트 350 은 위키 넷만 뺐고, 이 팔은 위키 다섯과 검색 하나를 한꺼번에 되살린다. 그러나 방향은 같리지 않는다. 그리고 이것이 명제를 살린다: 통제로 쓰기에 옳은 열이 판에서는 해로운 열이라는 것은 모순이 아니다 — 두 물음이 다르다.

4. 못 한 것

(1) 없어진 §H·§E 를 바이트로 복원하지 못했다 — 원본이 git 어디에도 없다. 앞값이 같다는 것은 다시 쓴 코드가 같은 수를 낸다는 뜻이고 같은 코드였다는 뜻이 아니다. (2) 앞 노트가 자기 신고한 항진명제(배선 W6)를 또 안 고쳤다 — 찾았고 게시했고 두 사이클째 남아 있다. (3) 들뜸을 유보로 갈라 예측 성능으로 재지 못했다 — 이 노트의 모든 ρ 는 여전히 연관이다. (4) 파수병은 `np.full/np.where` 만 본다 — $v[\sim ok] = 0.5$ 폴의 직접 대입 채움은 못 본다. 「안 돈 줄」은 「원리상 안 돈다」가 아니라 「이 주행에서 안 돌았다」다. (5) 판 팔의 도메인별 Δ 를 안 남겼다 — 팔을 짤 함수가 씨앗마다 묶음 점수만 저장하고 도메인별 점수를 버렸다. 앞 노트의 판 팔은 그것을 실었는데 이 노트는 못 싣는다. 다시 돌리면 30 분이라 이 사이클에서는 안 돌렸다 — 없는 것을 있는 것처럼 적지 않는다.